INSTRUCTION SET

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS

Instruction Set of Assembly Language

Instruction Set Definition: A set of instructions a processor can execute.

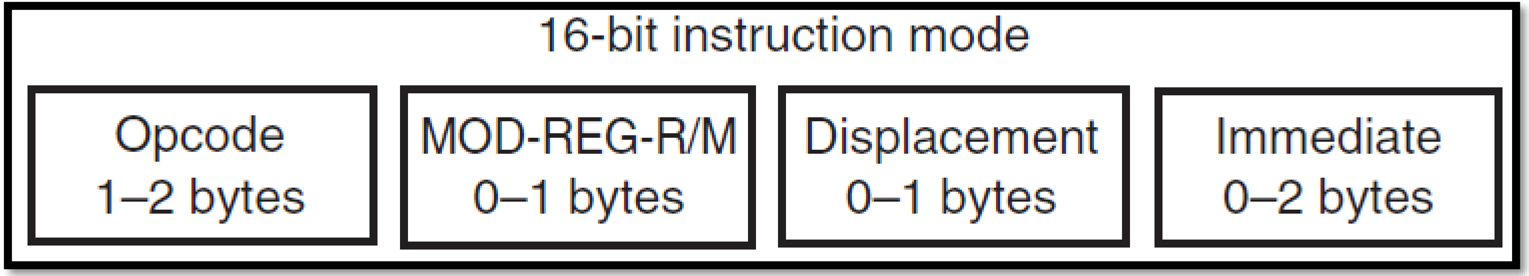
Processor-Specific ISA: Each processor has its unique Instruction Set Architecture (ISA).

Assembly Language: Human-readable representation of machine code for the ISA.

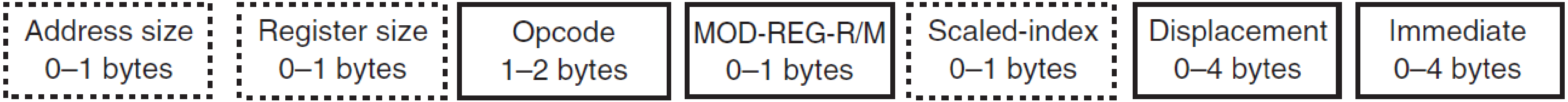
Software-Hardware Interface: ISA bridges software and hardware for effective communication.

Benefits of ISA: Standardization, compatibility, performance, and innovation in computing.

Instruction Format



32-bit instruction mode (80386 through Pentium 4 only)



Instruction Format

- Opcode
- First 6 bits are binary opcode
- 1 bit is Direction

D=1, data to the REG field

D=0, data flow from REG field

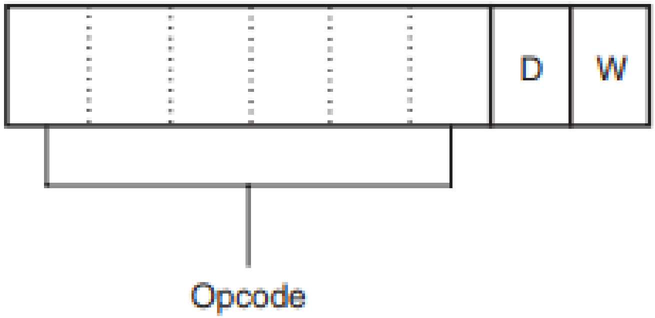


Fig. Byte 1 of Opcode

Direction BIT

D=1 , if data is moved **TO register** identified by REG field

D=0, if instruction is moving data **FROM register** mentioned in REG field

Instruction Format

Opcode

- First 6 bits are binary opcode
- 1 bit is Word
- W=1 , data size is word or doubleword
- W=0, data size is always a byte

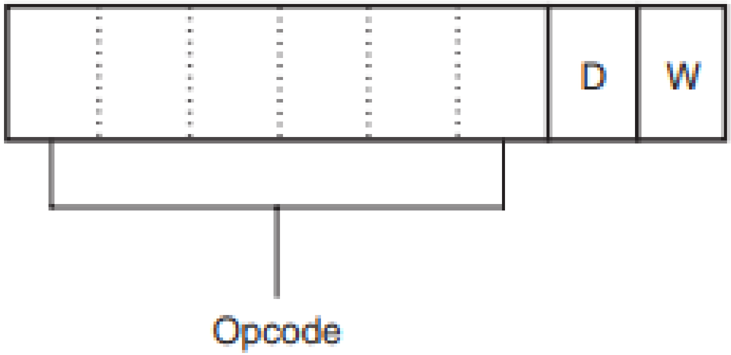


Fig. Byte 1 of Opcode

Instruction Format

Byte 2 of instruction

- MOD – specifies addressing mode for the selected instruction
- 11- Register addressing mode
- 00,01,10 – Data memory addressing modes

E.g. MOV AL, [DI] – No displacement

MOV AL,[DI+2] – 8 bit displacement

MOV AL,[DI+1000H]- 16 bit displacement

MOD	*Function*
00	No displacement
01	8-bit sign-extended displacement
10	16-bit signed displacement
11	R/M is a register

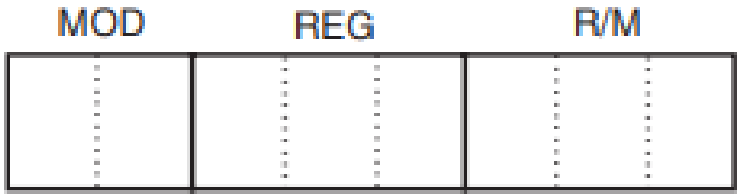


Fig. Byte 1 of Instruction

Sign Extension

Sign bit of a number is used to extend the bit-width of the number while preserving its sign.

- Necessary when working with data of different sizes or when performing operations that involve different-sized operands.

8 bit displacement are sign extended to 16 bit displacements
E.g. 00H-7FH
Extended to 0000H -007FH

E.g. 80H- FFH
Extended to FF80H-FFFH

Instruction Format

Byte 2 of instruction

- REG – registers assigned for the instruction

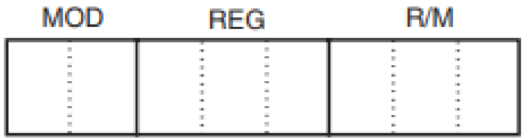


Fig. Byte 1 of Instruction

Code	W = 0 *(Byte)*	W = 1 *(Word)*	W = 1 *(Doubleword)*
000	AL	AX	EAX
001	CL	CX	ECX
010	DL	DX	EDX
011	BL	BX	EBX
100	AH	SP	ESP
101	CH	BP	EBP
110	DH	SI	ESI
111	BH	DI	EDI

Instruction Format

Byte 2 of instruction

- R/M – If the MOD field contains 00, 01, or 10 for 16 bit instructions

R/M Code	*Addressing Mode*
000	DS:[BX+SI]
001	DS:[BX+DI]
010	SS:[BP+SI]
011	SS:[BP+DI]
100	DS:[SI]
101	DS:[DI]
110	SS:[BP]*
111	DS:[BX]

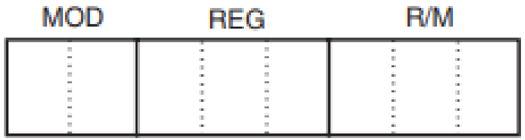


Fig. Byte 1 of Instruction

Instruction Format

R/M \ MOD	00	01	10	11	
				W = 0	W = 1
000	[BX] + [SI]	[BX] + [SI] + d8	[BX] + [SI] + d16	AL	AX
001	[BX] + [DI]	[BX] + [DI] + d8	[BX] + [DI] + d16	CL	CX
010	[BP] + [SI]	[BP] + [SI] + d8	[BP] + [SI] + d16	DL	DX
011	[BP] + [DI]	[BP] + [DI] + d8	[BP] + [DI] + d16	BL	BX
100	[SI]	[SI] + d8	[SI] + d16	AH	SP
101	[DI]	[DI] + d8	[DI] + d16	CH	BP
110	d16 (direct address)	[BP] + d8	[BP] + d16	DH	SI
111	[BX]	[BX] + d8	[BX] + d16	BH	DI
	MEMORY MODE			REGISTER MODE	

d8 = 8-bit displacement d16 = 16-bit displacement

innovate achieve lead

Instruction Format

E.g.

2 Byte Instruction – 8BECH

Binary – 1000 1011 1110 1100

Instruction Format

E.g.
2 Byte Instruction – 8BECH
Binary – 1000 1011 1110 1100

- Opcode – 100010 (MOV Instruction)
- D and W = 1 -> word moves into destination register specified in the REG field
- REG filed = 101 -> register BP
- MOD=11 -> R/M is register
- R/M = 100 (SP)
- -> Instruction moves data from SP into BP -> MOV BP, SP

Instruction Format

Opcode						D	W
1	0	0	0	1	0	1	1

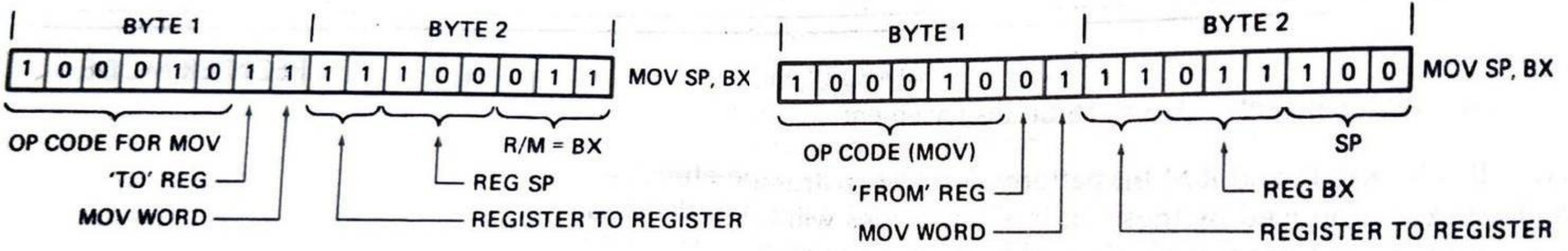
MOD		REG			R/M		
1	1	1	0	1	1	0	0

Opcode = MOV
D = Transfer to register (REG)
W = Word
MOD = R/M is a register
REG = BP
R/M = SP

Direction BIT

D=1 , if data is moved **TO register** identified by REG field

D=0, if instruction is moving data **FROM register** mentioned in REG field



innovate achieve lead

Example 2

What is the machine code for

MOV AX, BX

Example

What is the machine code for

MOV AX, BX

- Opcode- 100010
- D=1
- W= 1
- REG= 000
- MOD=11
- R/M= 011
- Instruction = 1001011 00011011

Special Addressing Mode

MOV [1000H],DL

MOV NUMB,DL

MOD= ?

R/M = ?

Special Addressing Mode

MOV [1000H],DL

MOV NUMB,DL

MOD= 00

R/M = 110

Assembler takes care of this by using an 8-bit displacement (MOD=01) of 00H

[BP+0]=[BP]

MOV [1000H],DL

Opcode						D	W
1	0	0	0	1	0	0	0

Byte 1

MOD		REG			R/M		
0	0	0	1	0	1	1	0

Byte 2

Displacement—low							
0	0	0	0	0	0	0	0

Byte 3

Displacement—high							
0	0	0	1	0	0	0	0

Byte 4

Opcode = MOV
D = Transfer from register (REG)
W = Byte
MOD = because R/M is [BP] (special addressing)
REG = DL
R/M = DS:[BP]
Displacement = 1000H

MOV [BP],DL

Opcode						D	W
1	0	0	0	1	0	0	0

Byte 1

MOD		REG			R/M		
0	1	0	1	0	1	1	0

Byte 2

8-bit displacement							
0	0	0	0	0	0	0	0

Byte 3

Opcode = MOV
D = Transfer from register (REG)
W = Byte
MOD = because R/M is [BP] (special addressing)
REG = DL
R/M = DS:[BP]
Displacement = 00H

Example 3

What is the machine code for

ADD [BX+DI+1234H], AX

Opcode for ADD is 000000.

$$ADD \quad Operand_1 \; , \; Operand_2$$

$$ADD \quad \begin{array}{ccc} Reg. & , & Mem. \\ Mem. & , & Reg. \\ Reg. & , & Reg. \\ Mem. & , & Immed. \\ Reg. & , & Immed. \end{array}$$

$$Operand_1 \leftarrow Operand_1 + Operand_2$$

Output of ADD instruction will be stored in memory address hence D=0. AX is source operand (REG). Since data is moving from REG, D=0.

innovate achieve lead

THANK YOU